Datos complejos: Tuplas y Sets

Tuplas

Las **tuplas** son una de las estructuras de datos fundamentales en Python. Al igual que las listas, son **colecciones ordenadas** de elementos. Sin embargo, a diferencia de las listas, **las tuplas son inmutables**, lo que significa que una vez creadas, **no pueden ser modificadas**: no es posible agregar, eliminar ni cambiar elementos de una tupla.

Esta característica de inmutabilidad las convierte en estructuras **más seguras y eficientes** en cuanto a uso de memoria y velocidad, especialmente cuando se necesita garantizar que los datos no serán alterados accidentalmente.

Característica	Detalle
Ordenadas	Sí. Conservan el orden de los elementos.
Mutables	No. Son inmutables.
Indexables	Sí. Se puede acceder a los elementos por índice.
Iterables	Sí. Se pueden recorrer con **for**, **while**, etc.
Permiten duplicados	Sí. Pueden tener elementos repetidos.
Tipos heterogéneos	Sí. Pueden mezclar distintos tipos de datos (enteros, strings, booleanos, etc.).

Las **tuplas** se declaran utilizando **paréntesis ()** y separando los elementos por comas **,**. Al igual que con las listas, los elementos pueden ser de cualquier tipo de dato (números, cadenas, booleanos, etc.), e incluso contener otras tuplas o listas dentro.

```
tupla_peugeot = ("208", "2008", "308", "3008")
tupla_mixta = ("208", 1.6, 180, False)
tupla_con_listas = ("208", ["Rojo", "Azul", "Blanco", "Negro"], [1.4, 1.6], [159, 180], False)
```

Como observamos en el ejemplo de declaraciones de tuplas, podemos incluir elementos de distinto tipo, incluso listas.

Para que Python interprete que es una tupla con un único valor, **es obligatorio agregar una coma**, de lo contrario será considerado un dato común encerrado entre paréntesis:

```
tupla_de_un_valor = ("valor",)
```

Python permite declarar tuplas **sin paréntesis** si los elementos están separados por comas:

```
tupla_peugeot = "208", "2008", "308", "3008"
```

Al igual que las listas, las tuplas en Python son **estructuras indexadas**. Esto significa que podemos acceder a cada elemento utilizando su **índice** (posición), empezando desde 0. Las tuplas se pueden recorrer con un ciclo **for**, como cualquier secuencia.

Desempaquetado de tuplas

El **desempaquetado** consiste en extraer los valores de una **tupla** (o cualquier otro iterable como listas o conjuntos) y asignarlos directamente a **variables individuales** en una sola línea de código. Esto permite evitar múltiples líneas de asignación manual y hace que el código sea más legible y conciso.

En lugar de acceder a los elementos uno por uno utilizando índices, Python permite tomar todos los valores de la tupla y distribuirlos automáticamente en distintas variables, **respetando el orden de los elementos**.

Este mecanismo es muy útil cuando se trabaja con estructuras de datos donde cada posición tiene un significado específico (por ejemplo: nombre, apellido, edad). Además, el desempaquetado se puede utilizar en ciclos, funciones que retornan múltiples valores, y en combinaciones más avanzadas con el operador ***** para capturar múltiples elementos en una sola variable.

```
tupla_con_listas = ("208", ["Rojo", "Azul", "Blanco", "Negro"], [1.4, 1.6], [159, 180], False)
modelo, colores, motores, potencias, importado = tupla_con_listas

print(modelo)
print(colores)
print(motores)
print(potencias)
print(importado)
```

```
208
['Rojo', 'Azul', 'Blanco', 'Negro']
[1.4, 1.6]
[159, 180]
False
```

Métodos disponibles

A diferencia de las listas, **las tuplas son inmutables**, lo que significa que **una vez creadas no se pueden modificar**: no se pueden agregar, eliminar o cambiar elementos. Esta característica limita las operaciones que se pueden hacer directamente sobre ellas. Sin embargo, Python proporciona **dos métodos útiles** que permiten consultar información sobre los elementos contenidos en una tupla:

- **count():** Este método **cuenta cuántas veces aparece un valor determinado** dentro de la tupla. Es útil para saber la frecuencia de un dato específico.

```
tupla_numeros = (2, 4, 2, 7, 3, 11)
tupla_numeros.count(2)
```

- **index(valor):** Este método **devuelve el índice de la primera aparición del valor indicado**. Si el valor no está presente, se lanza un error.

```
tupla_numeros = (2, 4, 2, 7, 3, 11)
tupla_numeros.index(7)
```

Tuplas anidadas

Una **tupla anidada** es una tupla que contiene **otras tuplas como elementos**. Esta estructura permite representar datos más complejos y jerárquicos, como si fueran filas y columnas de una tabla, coordenadas, registros combinados, entre otros.

Las tuplas anidadas son útiles cuando queremos **mantener relaciones entre elementos** y al mismo tiempo **asegurar que esos datos no se modifiquen**.

```
tupla_con_listas = ("208", ("Rojo", "Azul", "Blanco", "Negro"), [1.4, 1.6], [159, 180], False)
```

Por ejemplo, antes teníamos que los colores dentro de la tupla eran una lista, la cual era mutable. Si yo quisiera que se fueran una tupla inmutable se cambian los corchetes por paréntesis. Y para acceso a cada elemento de la tupla interior se usa doble corchetes.

Conversión entre listas y tuplas

En Python, las **listas** y las **tuplas** son tipos de estructuras muy similares en cuanto a su contenido: ambas pueden almacenar múltiples elementos heterogéneos (de distintos tipos) y permiten acceso por índices.

La diferencia clave es que **las listas son mutables** y **las tuplas son inmutables**.

A veces, es útil convertir entre una y otra según lo que necesitemos en cada parte del programa.

- **Lista a Tupla:** Cuando tenemos una lista pero queremos proteger su contenido para que **no se modifique**, podemos convertirla a una tupla utilizando la función incorporada **tuple().**

```
lista = ["a", "b", "c"]
tupla = tuple(lista)
```

Esto es útil, por ejemplo, cuando queremos pasar una colección de datos que no debe ser alterada por otras partes del código.

- **Tupla a lista:** Si tenemos una tupla y queremos modificar sus valores, podemos convertirla a lista usando la función **list():**

```
tupla = ("a", "b", "c")
lista = list(tupla)
```

Este tipo de conversión es común cuando se trabaja con datos constantes que luego necesitan ser editados o extendidos.

Estas conversiones **no modifican el original**, sino que **crean una nueva estructura**.

Conjuntos (Sets)

En Python, los **conjuntos (set)** son una estructura de datos **dinámica**, **mutable** y **no ordenada**, utilizada principalmente para **almacenar elementos únicos**, es decir, **sin duplicados**.

Los conjuntos se inspiran en la teoría de conjuntos de la matemática, por lo que permiten realizar operaciones como **unión**, **intersección**, **diferencia**, y **diferencia simétrica** de manera directa y eficiente.

Algunas características fundamentales de los conjuntos en Python:

- No mantienen el orden de inserción.
- No permiten elementos duplicados.
- Solo pueden contener elementos **inmutables** (por ejemplo: números, strings, tuplas).
- Son útiles para eliminar duplicados de listas o comparar elementos entre colecciones.

Hay dos formas principales de declarar un conjunto:

1. Con llaves y todos sus elementos separados por comas. Siempre que no esté vacío.

```
numeros = {2, 8, 4, 5}
```

2. Con constructor, especialmente cuando se necesita crear uno vacío.

```
numeros = set()
```

No usar llaves vacías para crear un set vacío ya que eso creará un diccionario y no un set

Teoría de conjuntos (matemáticas)

La **teoría de conjuntos** es una rama fundamental de la matemática que se encarga de estudiar **colecciones de objetos**, llamados **elementos**. Estas colecciones reciben el nombre de **conjuntos**.

Un **conjunto** es una agrupación bien definida de objetos distintos, llamados elementos o miembros del conjunto.

Se representa usualmente con **llaves {}** o con una **letra mayúscula**, y sus elementos se separan por comas. Ejemplo:

A = {2, 4, 6, 8}

Este conjunto A contiene los números 2, 4, 6 y 8.

Propiedades básicas de los conjuntos

- **Elementos únicos:** En un conjunto, **no hay elementos repetidos**.
  {1, 2, 2, 3} es equivalente a **{1, 2, 3}.**

- **Sin orden:** El orden en que se enumeran los elementos no importa.
  {1, 2, 3} = **{3, 2, 1}**

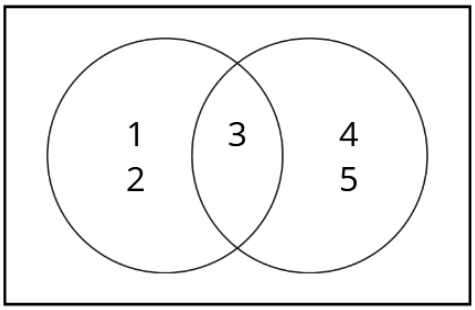
- **Pertenencia:** Para indicar que un elemento pertenece o no a un conjunto se usan símbolos:

  a. **∈** significa "pertenece a".

  b. **∉** significa "no pertenece a".

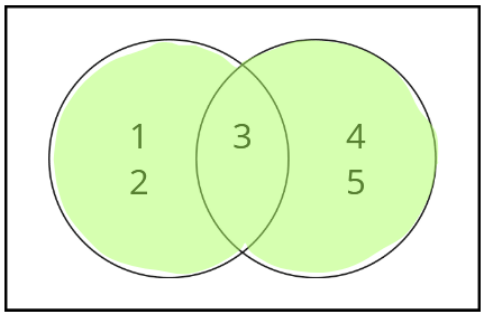
  c. Ejemplo: **4 ∈ A** y **5 ∉ A.**

Operaciones fundamentales

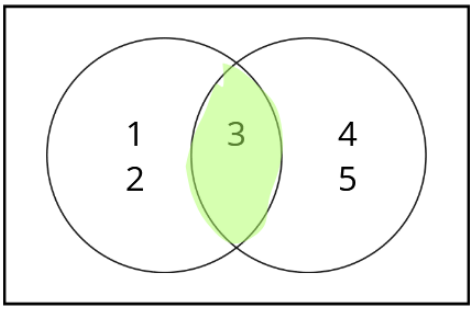


1. **Unión (U):**
   Combina todos los elementos de dos conjuntos.
   Ejemplo:

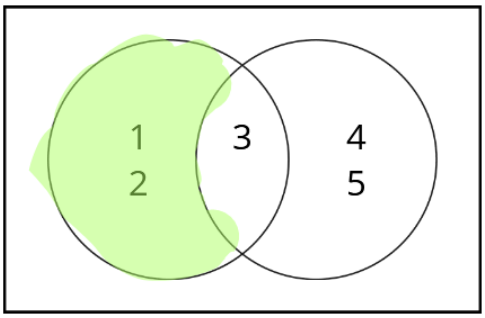
Si **A = {1, 2, 3} y B = {3, 4, 5}**
→ **A U B = {1, 2, 3, 4, 5}**



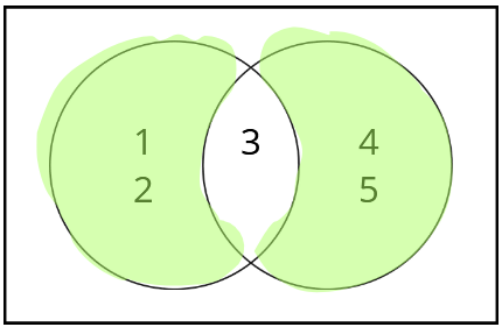
2. **Intersección (∩):**
   Elementos que están en **ambos** conjuntos.
   → **A ∩ B = {3}**



3. **Diferencia (−):**
   Elementos que están en un conjunto pero no en el otro.
   → **A − B = {1, 2}**



4. **Diferencia simétrica (△ o ⊕):**
   Elementos que están en uno u otro, pero **no en ambos**.
   → **A △ B = {1, 2, 4, 5}**



Conjuntos especiales

- **Conjunto vacío (∅):** No contiene elementos.
  → **∅ = {}**

- **Conjunto universal (U):** Contiene todos los elementos posibles bajo un cierto contexto.

Aplicación en Python

La **estructura set en Python** surge como una representación directa de los conjuntos matemáticos, heredando sus principios más importantes: **no repetición de elementos, ausencia de orden** y **posibilidad de realizar operaciones de conjunto** como unión, intersección o diferencia.

Este tipo de dato resulta especialmente útil cuando en un programa necesitamos **almacenar datos sin duplicados** y además **compararlos o analizarlos** desde una lógica de pertenencia o de cruce entre grupos.

Por ejemplo, puede ser útil para:

- Detectar si un elemento ya fue registrado en una colección.
- Comparar listas de usuarios activos e inactivos.
- Eliminar automáticamente valores repetidos en una gran cantidad de datos.
- Realizar filtros rápidos entre diferentes conjuntos de información.

La ventaja de los **set** es que están **optimizados internamente** para este tipo de tareas, lo que los hace muy eficientes en comparación con otras estructuras cuando se trata de búsquedas o comparaciones.

Métodos de Conjuntos

Al igual que las listas, los **conjuntos en Python** cuentan con una serie de **métodos integrados** que permiten manipular, consultar o combinar los datos que contienen. Estos

métodos no necesitan ser definidos por el usuario, ya que forman parte de la **biblioteca estándar del lenguaje**.

Cuando aplicamos un método a un conjunto, en realidad estamos invocando una **función que pertenece al objeto set**. Esta función realiza una operación específica sobre el contenido del conjunto, como agregar un nuevo elemento, eliminar uno existente o comparar conjuntos entre sí.

Una diferencia importante con las listas es que los conjuntos, al no tener un orden definido, **no permiten acceder a sus elementos mediante índices** ni tampoco ordenarlos directamente. Sin embargo, sus métodos están muy bien adaptados para realizar operaciones **matemáticas de conjuntos**, lo que los vuelve extremadamente útiles en ciertas situaciones.

Método add()

El método **add()** se utiliza para **agregar un único elemento** a un conjunto. Si el elemento ya existe, **no se agrega nuevamente**, ya que los conjuntos **no permiten duplicados**.

```
conjunto = {1, 3, 5}
conjunto.add(7)
```

- Solo permite agregar **un valor por vez**.
- Si se intenta agregar un elemento que **ya está presente**, no se produce ningún cambio ni error.
- El elemento puede ser de cualquier tipo **inmutable** (por ejemplo, **int, str, tuple)**. No se pueden agregar listas ni otros conjuntos porque son **mutables** y no hashables.
- En caso de que se agregue un tupla al conjunto, ésta se agrega como un único elemento del conjunto.

```
conjunto = {1, 3, 5}
conjunto.add((7,9,11))
```

```
{1, 3, 5, (7, 9, 11)}
```

Método remove()

El método **remove() elimina un elemento específico** de un conjunto. Si el elemento **no existe**, Python lanza un error (**KeyError**).

```
conjunto.remove(3)
```

- Elimina **exactamente el valor indicado**, si está presente.
- Si el valor **no está** en el conjunto, el programa **lanza un error**.
- Sirve cuando estás **seguro de que el elemento existe** en el conjunto.

Método discard()

El método **discard() elimina un elemento específico** de un conjunto **si está presente**.

A diferencia de **remove()**, **no lanza un error** si el elemento no existe.

```
conjunto.discard(3)
```

Método clear()

El método **clear() elimina todos los elementos del conjunto**, dejándolo vacío.

Es una forma directa de **reiniciar** un conjunto sin tener que crear uno nuevo.

```
conjunto.clear()
```

- Cuando queremos **vaciar completamente un conjunto** para reutilizarlo desde cero.
- Es útil en situaciones donde una colección estática de datos deja de ser relevante en cierto punto del programa, y se quiere **liberar o reiniciar el espacio** usado.

Después de usar **clear()**, el conjunto no es **None** ni se elimina, sigue existiendo, pero **sin ningún elemento**.

Método pop()

El método **pop() elimina y devuelve un elemento arbitrario** del conjunto.

Esto significa que **no se puede predecir cuál será el elemento eliminado**, ya que los conjuntos no tienen orden.

- Cuando necesitamos **reducir progresivamente un conjunto** trabajando con los elementos sin importar el orden.
- Puede servir como una forma básica de **recorrer y vaciar un conjunto**.
- Si el conjunto está vacío, usar **pop() provoca un error** (**KeyError**).

- Como el orden es arbitrario, **no se recomienda este método si importa qué elemento se elimina**.

```
elemento = conjunto.pop()
```

Método update()

El método **update() agrega múltiples elementos a un conjunto** a partir de **otro iterable** (como otro conjunto, una lista, una tupla, etc.).

No agrega duplicados: solo los elementos nuevos que aún no estaban.

```
conjunto.update([7, 9, 11])
```

- **Modifica el conjunto original en lugar de crear uno nuevo.**
- Es útil cuando se quiere **unir o extender** el conjunto con varios elementos a la vez.
- El elemento iterable que se agrega se descompone y se agregan como elementos separados dentro del conjunto.

```
{1, 3, 5, 7, 9, 11}
```

Método union()

El método **union() devuelve un nuevo conjunto** que contiene **todos los elementos de dos o más conjuntos**, **sin duplicados**. Equivale a la **unión matemática** de conjuntos.

```
conjunto_impares = {1, 3, 5, 7}
conjunto_pares = {2, 4, 6}
conjunto_union = conjunto_impares.union(conjunto_pares)
```

- Para **combinar datos** de varias fuentes sin preocuparse por repeticiones.
- Ideal en situaciones donde se necesita una **colección única** de elementos provenientes de múltiples listas, conjuntos, o entradas del usuario.
- El método no modifica los conjuntos originales, sino que devuelve uno nuevo.
- Todos los elementos repetidos se eliminan automáticamente (como en todo conjunto).

También se puede usar el operador **|** (pipe, dependiendo de la bibliografía)

```
conjunto_impares = {1, 3, 5, 7}
conjunto_pares = {2, 4, 6}
conjunto_union = conjunto_impares | conjunto_pares
```

Método intersection()

El método **intersection()** devuelve un nuevo conjunto con los elementos que **son comunes** a **dos o más conjuntos**. Equivale a la **intersección matemática** entre conjuntos.

```
conjunto_pares = {2, 4, 6, 8}
conjunto_pares_2 = {6, 8, 10, 12}
conjunto_intersection = conjunto_pares.intersection(conjunto_pares_2)
```

- Para identificar elementos **compartidos** entre varios conjuntos.
- Útil en análisis de datos, filtros cruzados, coincidencias entre listas, etc.
- El método no modifica los conjuntos originales.
- Si no hay elementos en común, devuelve un conjunto vacío: **set()**.

También se puede usar el operador **&** (ampersand)

```
conjunto_pares = {2, 4, 6, 8}
conjunto_pares_2 = {6, 8, 10, 12}
conjunto_intersection = conjunto_pares & conjunto_pares_2
```